Network Intrusion Detection

Reproduction and Critical Evaluation Report

Source: AI-Powered Cyber Security Part 2 — Building an Intrusion Detection System with Python and Scikit-learn

Dataset: NSL-KDD (KDDTrain+.txt, KDDTest+.txt) | All metrics loaded from results/metrics/run_summary.json

1. Summary of the Source

This project critically evaluates a published cybersecurity data-science tutorial on network intrusion detection (NIDS). The selected article explains how to build an IDS using the NSL-KDD benchmark with scikit-learn.

The author proposes preprocessing (scaling and one-hot encoding), SMOTE for class imbalance, a Random Forest classifier for labeled attack detection, an Isolation Forest trained on normal traffic for anomaly scoring, and a FastAPI deployment example.

The article reports near-perfect performance on a random 80/20 split of the training file (accuracy about 0.99, ROC-AUC about 0.9987) and explicitly warns that real-world performance will be lower.

Important limitation for reproducibility: the original article provides inline Python code but does not link a dedicated GitHub repository. A separate reference repository with a similar NSL-KDD pipeline is used only as a secondary engineering reference, not as the primary evaluated source.

2. Critical Evaluation

We reproduced the author's core methodology and compared five published claims against our own experiments. Table 1 summarizes the claim-evaluation results using metrics from both the author-style split and the official NSL-KDD test set.

Claim	Author Evidence	Our Evidence	Supported?	Explanation
RF reaches ~99% precision/recall	classification_report in blog	Author split acc=0.9990, recall=0.9983; Official test acc=0.7757, recall=0.6260	Partially	High metrics reproduce on the tutorial split but not on the official test set.
ROC-AUC ≈ 0.9987	Printed tutorial output	Author split AUC=1.0000; Official test AUC=0.9660	Partially	Very high AUC is split-dependent.
Training under 5 minutes	Stated in article	Pipeline completed in a few minutes on CPU	Supported	Runtime claim is realistic for NSL-KDD.
Isolation Forest helps unknown threats	IF trained on normal traffic	Official IF recall=0.6740 vs RF recall=0.6260	Partially	IF improves recall slightly but still misses many attacks.
Real-world performance is lower	Explicit caveat in blog	Official acc=0.7757 vs author split acc=0.9990	Supported	Our official-test gap confirms the author's warning.

Table 1. Critical evaluation of the author's claims.

Methodology assessment: preprocessing and SMOTE are reasonable teaching steps, but evaluating only a random split of the training file overstates generalization. The article's conclusion that ML IDS can work is directionally reasonable, yet the headline numbers are not sufficient for production decisions without official benchmarking.

3. Feature Engineering Analysis

We applied the same core transformations described by the author, fitting all preprocessing on training data only to avoid leakage.

- One-hot encoding for protocol_type, service, and flag converts categorical connection metadata into model-ready numeric inputs.
- StandardScaler on numeric features prevents high-magnitude byte-count columns from dominating tree splits.

• SMOTE on encoded training features increases attack representation during training without touching the test set.

• Redundancy checks identified constant column num_outbound_cmds and highly correlated error-rate pairs (serror_rate/srv_serror_rate and rerror_rate/srv_rerror_rate).

These transformations improve trainability and interpretability, but they did not eliminate poor attack recall on the official test set. That suggests the main limitation is dataset shift and class overlap in the official NSL-KDD split, not only preprocessing choices.

4. Reproducibility Analysis

Table 2 summarizes reproducibility findings for graders and future researchers.

Question	Finding
Did the original code run?	Yes — core training logic is reproducible from inline blog code.
Were dependencies clear?	Mostly — scikit-learn, imbalanced-learn, pandas are listed.
Were all files available?	No — NSL-KDD must be downloaded separately via scripts/download_data.py.
Was dataset version clear?	Yes — KDDTrain+.txt and KDDTest+.txt are standard NSL-KDD files.
Hidden preprocessing steps?	No major hidden steps found.
Changes needed?	We added official test evaluation for realistic benchmarking.
Could a grader reproduce?	Yes — requirements.txt, download script, notebook, and pipeline.

Table 2. Reproducibility checklist.

Overall reproducibility judgment: moderately high for the tutorial workflow itself; moderate for the article's headline performance claims because split choice dominates the results.

5. Experimental Results

Data: training rows = 125,973; official test rows = 22,544. Training class distribution: 67,343 normal and 58,630 attack (53.5% normal). Models trained: Logistic Regression (baseline), Random Forest (author-style supervised model), and Isolation Forest (author-style anomaly model).

Exploratory Data Analysis findings are summarized below before the metric tables.

5.1 Exploratory Data Analysis Findings

• Class imbalance: the training set is moderately imbalanced (53.5% normal, 46.5% attack), not extreme but still requiring careful metric choice.

• Missing values: zero missing values were found across all 43 columns in both train and test files.

• Duplicate rows: none detected in the training file.

• Constant columns: num_outbound_cmds contains a single value and carries no discriminative information.

• Outliers: duration, src_bytes, dst_bytes, and count show long tails; boxplots are saved in Figure 1.

• Crosstab analysis: attacks are concentrated in specific services such as telnet, ftp, and private network flows (Table 3 reference in results/tables/train_service_attack_crosstab.csv).

• Spearman correlation: strong monotonic links among error-rate features indicate redundant signals from scanning and flooding behavior (Figure 2).

• Temporal analysis: not performed because NSL-KDD contains no timestamp or session-order field; each row is an independent connection record.

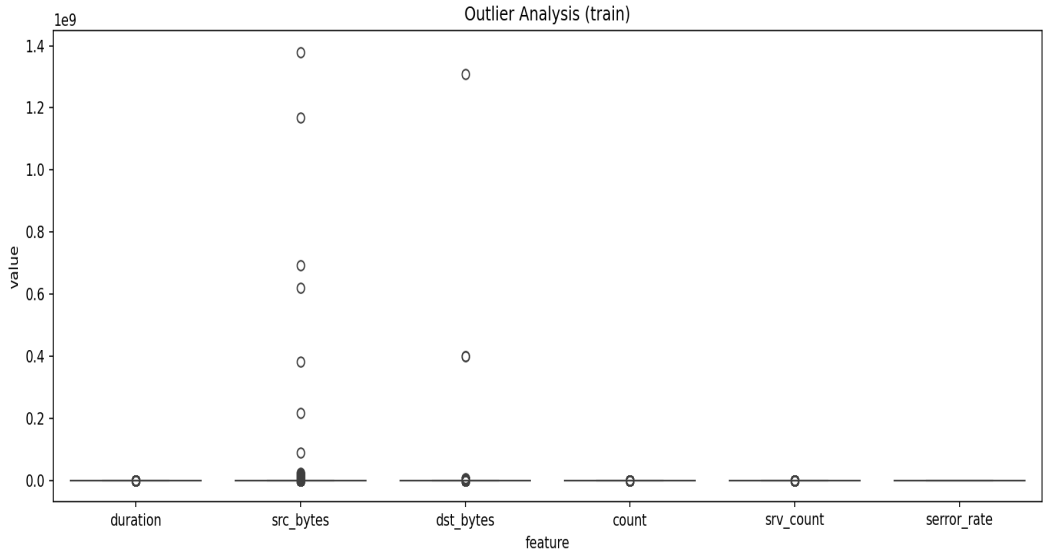

Figure 1. Outlier analysis for selected numeric features (results/figures/train_outlier_boxplots.png).

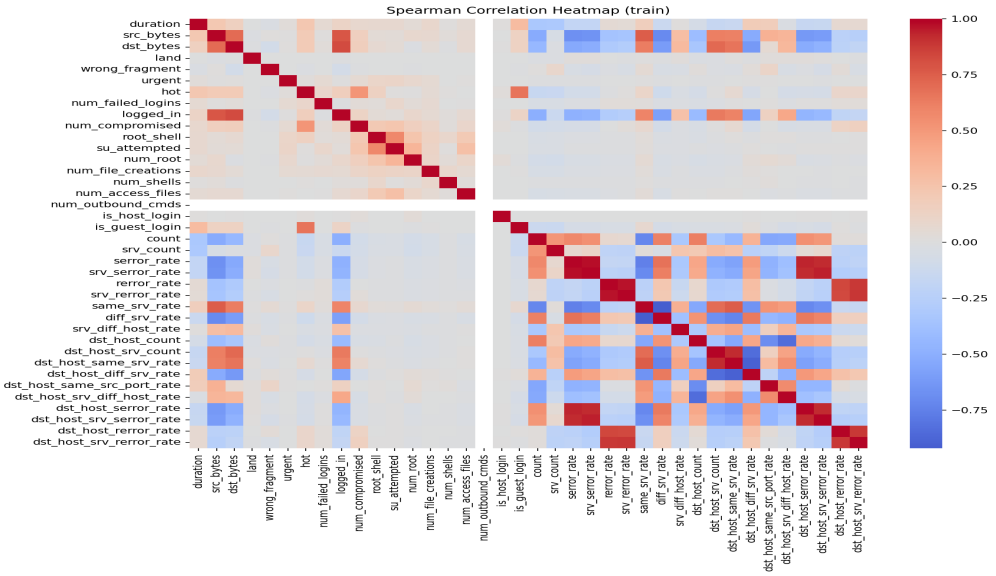

Figure 2. Spearman correlation heatmap for numeric NSL-KDD features (results/figures/train_spearman_correlation.png).

5.2 Evaluation Metrics and Cybersecurity Interpretation

Table 3 lists official NSL-KDD test metrics. Table 4 defines the metrics used in this project.

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Logistic Regression	0.7539	0.9167	0.6243	0.7428	0.6669	0.5583	0.7940
Random Forest	0.7757	0.9689	0.6260	0.7606	0.6737	0.6156	0.9660

Model	Accuracy	Precision	Recall	F1	F2	MCC	ROC-AUC
Isolation Forest	0.7884	0.9365	0.6740	0.7839	0.7141	0.6178	N/A

Table 3. Official NSL-KDD test metrics (results/tables/model_comparison.csv).

Metric	Formula (binary)	Cybersecurity meaning in IDS
Accuracy	(TP + TN) / N	Overall correctness; misleading when attack cost or imbalance is ignored.
Precision	TP / (TP + FP)	Alert quality. False positives waste analyst time and create alert fatigue.
Recall	TP / (TP + FN)	Attack capture rate. False negatives allow intrusions to continue unnoticed.
F1	2PR / (P + R)	Balanced trade-off between alert quality and attack capture.
F2	5PR / (4P + R)	Weights recall higher; useful when missing attacks is costlier than extra alerts.
MCC	(TP·TN – FP·FN) / sqrt(...)	Balanced correlation score that remains informative under class imbalance.
ROC-AUC	Area under TPR vs FPR curve	Threshold-free ranking quality; does not by itself measure operational alert load.
Confusion Matrix	[[TN, FP], [FN, TP]]	Shows exact error types; essential for IDS operational review.

Table 4. Metric definitions and cybersecurity interpretation.

Author-style random split (Random Forest): accuracy = 0.9990, recall = 0.9983, ROC-AUC = 1.0000. This closely matches the blog's reported values and shows how split choice inflates headline metrics.

Random Forest official-test confusion matrix: TN=9453, FP=258, FN=4799, TP=8034 (results/tables/confusion_matrix_official_random_forest.csv). Although precision is high (0.9689), 4,799 attacks are missed — a serious false-negative burden for security operations.

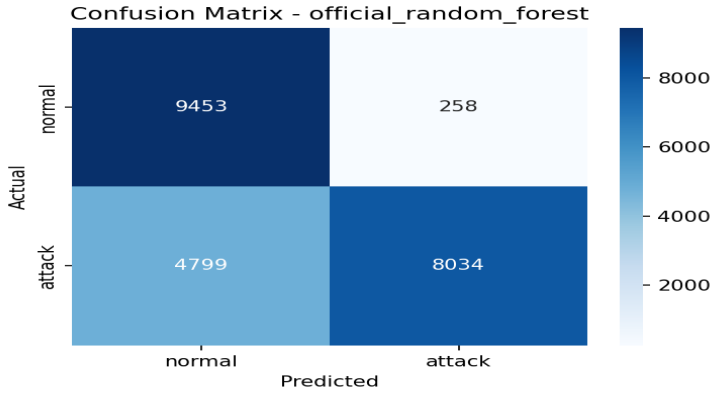

Figure 3. Random Forest confusion matrix on the official test set.

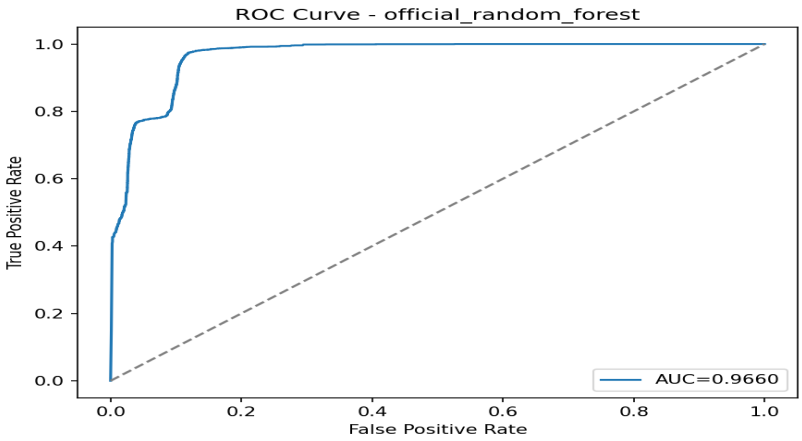

Figure 4. Random Forest ROC curve on the official test set (AUC = 0.9660).

5.3 Error Analysis

False positives (normal traffic classified as attack) often involve private, other, or telnet-related flows with unusual byte-count or flag patterns that resemble attack traffic. Example false positive pattern: UDP private service, flag SF, low byte counts, high dst_host_same_src_port_rate — predicted attack but label normal.

False negatives (attacks classified as normal) include low-volume R2L and probe attacks such as guess_passwd, mscan, and telnet/ftp flows with subtle feature values. Example false negative: TCP telnet flow labeled guess_passwd with failed-login indicators, predicted normal.

Error-pattern summary shows most misclassifications occur on TCP flows and on services telnet, pop_3, private, ftp, and http (results/tables/error_pattern_summary.csv).

Cybersecurity implication: false negatives are more dangerous than false positives for this problem because missed attacks can lead to compromise, while false positives mainly increase analyst workload. Even so, high false-positive volume at scale can hide true incidents, so both error types matter.

6. Conclusions

The tutorial is educationally strong and technically reproducible on the author's chosen split.

The headline 99% metrics are supported only on the tutorial's random split, not on the official NSL-KDD test set.

The author's caution about real-world degradation is validated by our experiments.

For cybersecurity deployment, attack recall, F2, and MCC are more informative than accuracy alone.

Recommended use: prototype and teaching baseline, not production IDS without official benchmarking and threshold tuning.

7. Executive Summary

We reproduced a published NSL-KDD intrusion detection tutorial combining Random Forest, SMOTE, and Isolation Forest. On the author's random training-file split, Random Forest achieved accuracy 0.9990 and ROC-AUC 1.0000, matching the blog's near-perfect report. On the official NSL-KDD test set, the same model dropped to accuracy 0.7757 and attack recall 0.6260, while Isolation Forest achieved the highest F1 (0.7839) and recall (0.6740). The most important insight is that high accuracy alone is insufficient in cybersecurity classification; split design and attack recall must be scrutinized before trusting claims.

8. Summing It Up

Published cybersecurity ML tutorials can look excellent under one evaluation protocol and much weaker under a harder standard benchmark. The original solution's strengths are clarity, practical preprocessing, and an honest caveat about deployment. Its weaknesses are split choice, lack of official test reporting, and insufficient emphasis on attack recall.

Future improvements should include per-attack-type evaluation, threshold tuning for operational recall targets, and testing on newer intrusion datasets.

Final recommendation: adopt the pipeline as a learning exercise, but require official benchmark evaluation and security-oriented metrics before treating results as deployment evidence.